

```

"""
Saber AI Santander – Tutor Virtual para el Saber 11 (Julio 26, 2026)
Colores: Amarillo Santander #FFCD00 | Verde Santander #00914C
"""

import streamlit as st
import pdfplumber
import re
from openai import OpenAI

# -----
# PAGE CONFIG
# -----
st.set_page_config(
    page_title="Saber AI Santander",
    page_icon="🦅",
    layout="wide",
    initial_sidebar_state="expanded",
)

# -----
# CUSTOM CSS (colores Bandera de Santander)
# -----
st.markdown(
    """
<style>
/* — Header banner — */
.header-banner {
    background: linear-gradient(135deg, #FFCD00 0%, #e6b800 100%);
    padding: 1.2rem 2rem;
    border-radius: 12px;
    margin-bottom: 1.5rem;
    display: flex;
    align-items: center;
    gap: 1rem;
    box-shadow: 0 4px 15px rgba(255,205,0,0.4);
}
.header-banner h1 {
    color: #1a1a1a;
    margin: 0;
    font-size: 1.9rem;
    font-weight: 800;
    letter-spacing: -0.5px;
}
    """
)

```

```

.header-banner p {
  color: #333;
  margin: 0;
  font-size: 0.95rem;
}

/* — Sidebar — */
[data-testid="stSidebar"] {
  background: #00914C !important;
}
[data-testid="stSidebar"] * {
  color: #fff !important;
}
[data-testid="stSidebar"] .stSelectbox label {
  color: #FFCD00 !important;
  font-weight: 700;
}

/* — Primary buttons — */
.stButton > button {
  background-color: #00914C !important;
  color: #fff !important;
  border: none !important;
  border-radius: 8px !important;
  font-weight: 700 !important;
  padding: 0.5rem 1.5rem !important;
  transition: background 0.2s;
}
.stButton > button:hover {
  background-color: #007a3d !important;
}

/* — Score pills — */
.pill-critico {
  background:#e74c3c; color:#fff;
  border-radius:20px; padding:4px 14px;
  font-weight:700; font-size:0.85rem;
  display:inline-block;
}
.pill-mejorar {
  background:#f39c12; color:#fff;
  border-radius:20px; padding:4px 14px;
  font-weight:700; font-size:0.85rem;
  display:inline-block;
}
.pill-bien {
  background:#00914C; color:#fff;

```

```

        border-radius:20px; padding:4px 14px;
        font-weight:700; font-size:0.85rem;
        display:inline-block;
    }

    /* — Chat bubbles — */
    .stChatMessage { border-radius: 12px !important; }

    /* — Info card — */
    .info-card {
        background: #f0faf5;
        border-left: 5px solid #00914C;
        padding: 1rem 1.2rem;
        border-radius: 8px;
        margin: 0.8rem 0;
    }
</style>
""",
    unsafe_allow_html=True,
)

# -----
# OPENAI CLIENT
# -----
client = OpenAI(api_key=st.secrets["OPENAI_API_KEY"])

# -----
# SESSION STATE DEFAULTS
# -----
for key, default in {
    "messages": [],
    "scores": {},
    "area": "🧮 Matemáticas",
}.items():
    if key not in st.session_state:
        st.session_state[key] = default

# -----
# SIDEBAR
# -----
with st.sidebar:
    st.markdown("## 🦅 Saber AI Santander")
    st.markdown("---")
    area = st.selectbox(
        "Selecciona tu módulo:",
        [
            "🇮🇹 Diagnóstico (Subir Resultados)",

```

```

        "🧮 Matemáticas",
        "📖 Lectura Crítica",
        "🔬 Ciencias Naturales",
        "⚖️ Sociales y Ciudadanas",
        "🇬🇧 Inglés",
        "🧠 Socioemocionales (Nuevo 2026)",
    ],
)

st.session_state["area"] = area
st.markdown("---")
st.markdown("🇨🇴 **Examen:** 26 de Julio, 2026")
st.markdown("📍 **Para estudiantes de Santander**")

if st.session_state["scores"]:
    st.markdown("---")
    st.markdown("**📈 Mis puntajes ICFES:**")
    for mat, score in st.session_state["scores"].items():
        if score < 45:
            badge = f'<span class="pill-critico">{score}</span>'
        elif score <= 65:
            badge = f'<span class="pill-mejorar">{score}</span>'
        else:
            badge = f'<span class="pill-bien">{score}</span>'
        st.markdown(f"{mat}: {badge}", unsafe_allow_html=True)

# -----
# HEADER
# -----
st.markdown(
    """
    <div class="header-banner">
        <div style="font-size:3rem">🦅</div>
        <div>
            <h1>Saber AI Santander</h1>
            <p>Tu tutor virtual para el Saber 11 · Bucaramanga & Santander · Exam
            </p>
        </div>
    </div>
    """,
    unsafe_allow_html=True,
)

# -----
# AREA METADATA
# -----
AREA_META = {
    "🧮 Matemáticas": {
        "competencias": ["Interpretación y representación", "Formulación y ejecuc

```

```

        "tip": "En Matemáticas, el ICFES 2026 evalúa cómo aplicas el razonamiento
    },
    "📖 Lectura Crítica": {
        "competencias": ["Identificar y ubicar información", "Relacionar e interp
        "tip": "Lectura Crítica mide tu capacidad de analizar textos en tres nive
    },
    "🔬 Ciencias Naturales": {
        "competencias": ["Indagación", "Explicación de fenómenos", "Uso del conoc
        "tip": "Ciencias Naturales integra Biología, Física y Química. El foco es
    },
    "⚖️ Sociales y Ciudadanas": {
        "competencias": ["Conocimiento", "Argumentación", "Multiperspectivismo"],
        "tip": "Sociales evalúa pensamiento crítico ciudadano: historia, geográfi
    },
    "🇬🇧 Inglés": {
        "competencias": ["Reading", "Listening", "Writing"],
        "tip": "Inglés evalúa tu nivel según el Marco Común Europeo (A1 a B+). Pr
    },
    "🧠 Socioemocionales (Nuevo 2026)": {
        "competencias": ["Procesos emocionales", "Regulación cognitiva", "Habilid
        "tip": "¡Nueva sección 2026! Evalúa autorregulación emocional, empatía y
    },
}

# -----
# HELPER: classify score
# -----

def classify(score: float) -> tuple[str, str]:
    if score < 45:
        return "🔴 Prioridad Crítica", "pill-critico"
    elif score <= 65:
        return "🟡 Por Mejorar", "pill-mejorar"
    else:
        return "🟢 Buen Nivel", "pill-bien"

# -----
# HELPER: extract scores from ICFES PDF text
# -----

AREA_KEYWORDS = {
    "Matemáticas": ["matemáticas", "matematicas", "math"],
    "Lectura Crítica": ["lectura crítica", "lectura critica", "lectura"],
    "Ciencias Naturales": ["ciencias naturales", "ciencias"],
    "Sociales y Ciudadanas": ["sociales", "ciudadanas", "competencias ciudadanas"],
    "Inglés": ["inglés", "ingles", "english"],
    "Global": ["global", "puntaje global", "total"],
}

```

```

def extract_scores_from_text(text: str) -> dict:
    """
    Tries to find numeric scores near area keywords.
    ICFES result PDFs typically list scores as integers 0-100.
    """
    scores = {}
    text_lower = text.lower()
    lines = text_lower.split("\n")

    for area_name, keywords in AREA_KEYWORDS.items():
        for kw in keywords:
            for line in lines:
                if kw in line:
                    numbers = re.findall(r"\b([1-9][0-9]?|100)\b", line)
                    if numbers:
                        # Take the first plausible score (0-100)
                        for n in numbers:
                            val = int(n)
                            if 0 < val <= 100:
                                scores[area_name] = val
                                break
                    if area_name in scores:
                        break
            if area_name in scores:
                break
    return scores

# -----
# SYSTEM PROMPT BUILDER
# -----

def build_system_prompt(area: str, scores: dict) -> str:
    scores_txt = ""
    if scores:
        lines = [f" - {k}: {v} ({classify(v)[0]})" for k, v in scores.items()]
        scores_txt = "\nPuntajes ICFES anteriores del estudiante:\n" + "\n".join(
            lines

        competencias = ""
        if area in AREA_META:
            comp_list = ", ".join(AREA_META[area]["competencias"])
            competencias = f"\nÁrea actual: {area}\nCompetencias evaluadas: {comp_list}

    return f"""Eres un tutor veterano del ICFES con 20 años de experiencia en Buc
Tu misión es preparar a los estudiantes para el Saber 11 del 26 de julio de 2026.

```

REGLAS ABSOLUTAS:

1. NUNCA des la respuesta directa primero. SIEMPRE haz una pregunta socrática par
2. Identifica la competencia ICFES que evalúa cada pregunta (ej: Argumentación, I
3. Usa ejemplos con referencias locales de Santander: la industria del calzado en
4. Tono: animador, directo, profesional. Como un buen profesor bumangués.
5. Si el estudiante tiene puntajes bajos (< 45), prioriza esas áreas con más énf
6. Para el módulo Socioemocionales (nuevo 2026), evalúa procesos emocionales, reg
7. Cuando respondas, primero menciona la competencia evaluada en negrita, luego h
8. Máximo 3 párrafos por respuesta para mantenerla concisa y clara.

```
{scores_txt}
{competencias}
```

Ejemplo de respuesta correcta:

```
***Competencia: Argumentación** - ¡Buena pregunta! Antes de explicarte, dime: si
"""
```

```
# -----
# — MODULE: DIAGNÓSTICO
# -----

if area == "🇨🇴 Diagnóstico (Subir Resultados)":
    st.subheader("🇨🇴 Analizador de Resultados ICFES")
    st.markdown(
        '<div class="info-card">Sube el PDF con tus resultados ICFES anteriores.
        "El tutor analizará tus puntajes y personalizará las sesiones de estudio.
        unsafe_allow_html=True,
    )

uploaded = st.file_uploader("Selecciona tu PDF de resultados ICFES", type=["p

if uploaded:
    with st.spinner("Leyendo tu PDF... un momento, parece 🦅"):
        try:
            full_text = ""
            with pdfplumber.open(uploaded) as pdf:
                for page in pdf.pages:
                    t = page.extract_text()
                    if t:
                        full_text += t + "\n"

            scores = extract_scores_from_text(full_text)

            if scores:
                st.session_state["scores"] = scores
                st.success("✅ ¡Puntajes detectados exitosamente!")

                cols = st.columns(len(scores))
                for i, (mat, score) in enumerate(scores.items()):
```

```

label, css = classify(score)
with cols[i]:
    st.markdown(
        f"<div style='text-align:center'>"
        f"<div style='font-size:2rem;font-weight:800;colo"
        f"<div style='font-size:0.8rem;font-weight:700'>{"
        f"<span class='{css}'>{label}</span>"
        f"</div>",
        unsafe_allow_html=True,
    )

st.markdown("---")
st.markdown("### 💡 Plan de Estudio Recomendado")
for mat, score in scores.items():
    label, _ = classify(score)
    if score < 45:
        st.error(f"🔴 **{mat}** ({score} pts): {label} - Dedi
    elif score <= 65:
        st.warning(f"🟡 **{mat}** ({score} pts): {label} - Pr
    else:
        st.success(f"🟢 **{mat}** ({score} pts): {label} - Ma

st.info("🌀 Selecciona un módulo en la barra lateral para com
else:
    st.warning(
        "⚠️ No se encontraron puntajes numéricos en el PDF. "
        "Asegúrate de subir el reporte oficial del ICFES. "
        "Puedes ingresar tus puntajes manualmente abajo."
    )
with st.expander("✎ Ingresar puntajes manualmente"):
    manual_scores = {}
    areas_list = list(AREA_META.keys())
    c1, c2 = st.columns(2)
    for i, a in enumerate(areas_list):
        col = c1 if i % 2 == 0 else c2
        label = a.split(" ", 1)[-1]
        val = col.number_input(label, 0, 100, 50, key=f"manua
        manual_scores[label] = val
    if st.button("💾 Guardar puntajes"):
        st.session_state["scores"] = manual_scores
        st.success("¡Puntajes guardados! Selecciona un módulo

except Exception as e:
    st.error(f"Error al leer el PDF: {e}")

else:
    # Show manual entry even without upload

```



```

with st.expander("✎ ¿No tienes el PDF? Ingresa tus puntajes manualmente"
    manual_scores = {}
    areas_list = ["Matemáticas", "Lectura Crítica", "Ciencias Naturales",
    c1, c2 = st.columns(2)
    for i, a in enumerate(areas_list):
        col = c1 if i % 2 == 0 else c2
        val = col.number_input(a, 0, 100, 50, key=f"m_{a}")
        manual_scores[a] = val
    if st.button("💾 Guardar puntajes"):
        st.session_state["scores"] = manual_scores
        st.success("¡Listo! Ahora selecciona un módulo en la barra lateral")

# -----
# — MODULE: TUTOR CHAT
# -----
else:
    meta = AREA_META.get(area, {})

    # Area header
    emoji = area.split()[0]
    area_name = area.split(" ", 1)[1]
    st.subheader(f"{area}")

    if meta:
        comp_str = " · ".join(meta["competencias"])
        st.markdown(
            f'<div class="info-card"><b>Competencias evaluadas:</b> {comp_str}<br>
            f'<small>💡 {meta["tip"]}</small></div>',
            unsafe_allow_html=True,
        )

    # Reset chat if area changed
    if st.session_state.get("last_area") != area:
        st.session_state["messages"] = []
        st.session_state["last_area"] = area

    # Greeting from tutor
    score_note = ""
    if st.session_state["scores"]:
        area_key = area_name.strip()
        for k, v in st.session_state["scores"].items():
            if k.lower() in area_key.lower() or area_key.lower() in k.lower():
                label, _ = classify(v)
                score_note = f" Veo que en esta área obtuviste **{v} puntos**"
                break

    greeting = (

```

```

        f";Bienvenido, futuro profesional santandereano! 🦅{score_note} "
        f"Estamos en **{area}** y hoy vamos a trabajar duro como los artesano
        f"Cuéntame: ¿qué tema o pregunta del ICFES te está generando más difi
    )
    st.session_state["messages"].append({"role": "assistant", "content": gree

# Display chat history
for msg in st.session_state["messages"]:
    with st.chat_message(msg["role"], avatar="🦅" if msg["role"] == "assistant"
        st.markdown(msg["content"])

# Chat input
if prompt := st.chat_input(f"Pregúntame sobre {area_name}..."):
    st.session_state["messages"].append({"role": "user", "content": prompt})
    with st.chat_message("user", avatar="🎓"):
        st.markdown(prompt)

    with st.chat_message("assistant", avatar="🦅"):
        with st.spinner("El tutor está pensando... ☕"):
            try:
                system_prompt = build_system_prompt(area, st.session_state["s
                response = client.chat.completions.create(
                    model="gpt-4o",
                    messages=[
                        {"role": "system", "content": system_prompt},
                        *[
                            {"role": m["role"], "content": m["content"]}
                            for m in st.session_state["messages"]
                        ],
                    ],
                    max_tokens=600,
                    temperature=0.7,
                )
                reply = response.choices[0].message.content
            except Exception as e:
                reply = (
                    f"⚠️ Hubo un problema de conexión con el tutor: `{e}`. "
                    "Verifica tu API key en los Secrets de Streamlit."
                )

            st.markdown(reply)
            st.session_state["messages"].append({"role": "assistant", "content": reply})

# Quick topic buttons
if meta and len(st.session_state["messages"]) <= 1:
    st.markdown("---")
    st.markdown("***⚡ Inicio rápido – Elige un tema:***")

```

```

        cols = st.columns(len(meta["competencias"]))
        for i, comp in enumerate(meta["competencias"]):
            if cols[i].button(comp, key=f"quick_{i}"):
                msg = f"Quiero practicar la competencia de **{comp}** para el ICF
                st.session_state["messages"].append({"role": "user", "content": m
                st.rerun()

# -----
# FOOTER
# -----
st.markdown("---")
st.markdown(
    "<center style='color:#888;font-size:0.8rem'>"
    "🦅 Saber AI Santander · Basado en los cuadernillos ICFES 2026 oficiales · "
    "Hecho con ❤️ para los estudiantes de Bucaramanga y Santander"
    "</center>",
    unsafe_allow_html=True,
)

```